Grouping and Analyzing of Darknet Scanners with Graph Embeddings

Nealson Setiawan

Who's that looking through your window?

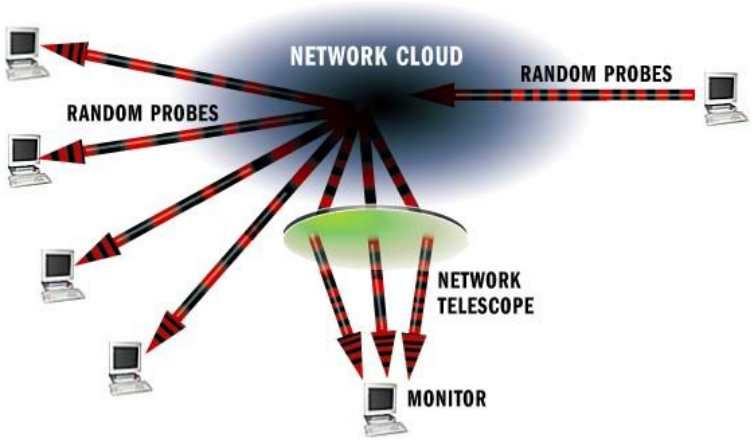
- Imagine you're doing your laundry
- You look out and someone's peering in your window
- Scanners are the same!
- They look into IPs find services there
- If only we can have a way to know what they're doing

Research Statement

Analyzing and interpretation of scanners’ behavior in Darknet data based on its scanning statistics by clustering on graph-ML embeddings.

Background: But what is the darknet anyway?

- The darknet are areas of **unused IP addresses** that have no known services on them
- Captured on network telescopes
  - they provide valuable data for researchers on scanners
- Scanning these areas historically have been:
  - botnet attacks, worms, and other malicious activities
- Not always suspicious
  - Accidentally misconfigure an IP address so it's not always sure to say they're malicious



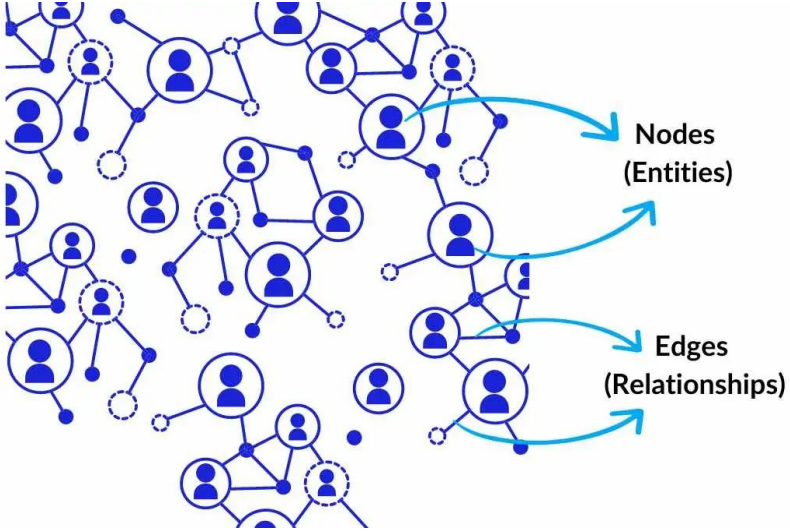
Need: Insights for an intrusion detection system

- Insights on darknet scanner behavior could lead to a better intrusion detection system
- For example, if a scanner tends to use UDP instead of TCP, then the IDS could improve UDP detection
- Scanning darknet gives IDS practitioners ideas on how to develop their IDS effectively
- Since similar IPs scanners potentially scan similar ports, graph-based methods could better aggregate similar IPs characteristics

Approach: Methodology

For this research, I propose a method for categorizing these data

1. **Construct a heterogenous graph** of the darknet data
2. Use a **graph neural network** to aggregate features from nearby nodes
3. **Get embeddings** for all the source nodes
4. **Cluster the embeddings** by optimizing silhouette score or other metrics



Approach: Other Works on Latent Space Clustering

Multiple other works have performed similar methodologies of projecting darknet features to a latent space then clustering them

- **DarkVec** uses word embeddings of source IPs to cluster
- **Poster** uses an auto-encoder to project features into a latent space before clustering
- **Detecting and Interpreting Changes in Scanning Behavior in Large Network Telescopes** also uses an auto-encoder to project features into a latent space before clustering

Approach: CAIDA Dataset

- UC San Diego's ongoing **network telescope** capturing horizontal scanning darknet data
  - since 2008 until today (ongoing)!
- Terabytes of data and researchers have used it to learn more about **DDoS, Botnets, and Worms**
- Each hour's data consists of **netflows of unique (source ip, protocol, destination port) pairs**

Approach: Dataset Features

- Source IP
- Number of destination ip addresses scanned
- Approximate number of flows: this should be the number of observed unique (destination_ip, source_port, protocol) pairs
- Number of packets sent
- Timestamp of first activity (within the hour)
- Timestamp of last activity (within the hour)
- Are all packets the same size (yes: 1, no: 0)
- Average packet size • Number of B {B | A.B.C.D in S }, where S= {UCSD.B.C.D | UCSD.B.C.D was scanned}
- {C | A.B.C.D in S }, where S= {UCSD.B.C.D | UCSD.B.C.D was scanned}
- {D | A.B.C.D in S }, where S= {UCSD.B.C.D | UCSD.B.C.D was scanned}
- Number of /24 blocks scanned
- Number of non-Conficker destinations scanned
- Destination Port

Approach: Graph Construction

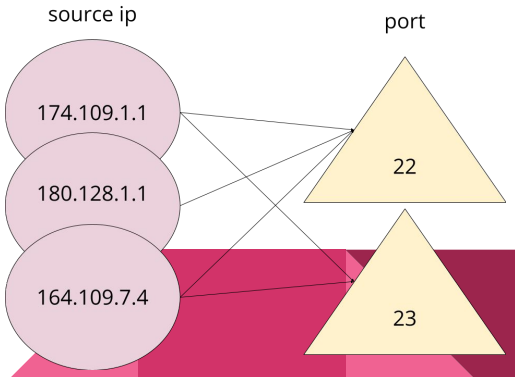
The graph is a heterogeneous graph (nodes and edges can have different types) with a source ip having an edge between each destination port it scans

Source Nodes Features

- Number of destination ip addresses scanned
- Approximate number of flows: this should be the number of observed unique (destination_ip, source_port, protocol) pairs
- Number of packets sent
- Average packet size
- Number of B {B | A.B.C.D in S }, where S= {UCSD.B.C.D | UCSD.B.C.D was scanned}
- {C | A.B.C.D in S }, where S= {UCSD.B.C.D | UCSD.B.C.D was scanned}
- {D | A.B.C.D in S }, where S= {UCSD.B.C.D | UCSD.B.C.D was scanned}
- Number of /24 blocks scanned
- Flag on whether the protocol used is TCP
- Flag on whether the protocol used is UDP

Destination node Features

- Destination Port number (OHE)



Approach: Training

- The model used is MetaPath2Vec (Meta's Node2Vec for heterogeneous graphs)
  - Trains by creating sequences of source nodes and destination ports and predicting the context nodes around each node
  - Model doesn't take edge features into account so no edge features
- Trained with Adam(lr=0.01) and batch size of 256 on Colab
- Trained for ~130,000 source nodes and 42 destination ports

Approach: Clustering

- To optimize for clustering, tested on three different clustering methods to optimize silhouette score
  - KMeans
  - HDBCSAN
  - Spectral Clustering
- KMeans with 7 clusters yield best results

Approach: Categories of Scanners

From this, 4 different categories of scanners are implied

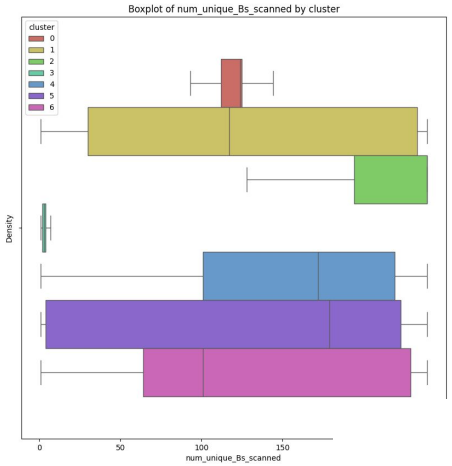
1. Traditional horizontal scanners
2. Subnet-based scanners
3. UDP scanners
4. Conficker scanners

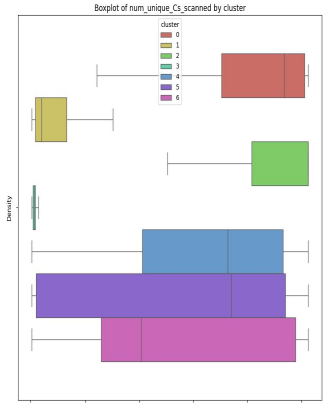
Scanners can be of multiple categories

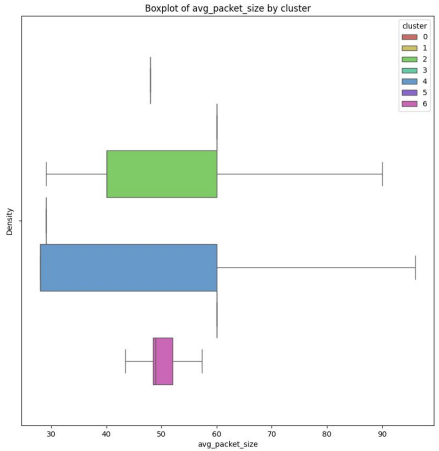
Approach: Traditional Scanners

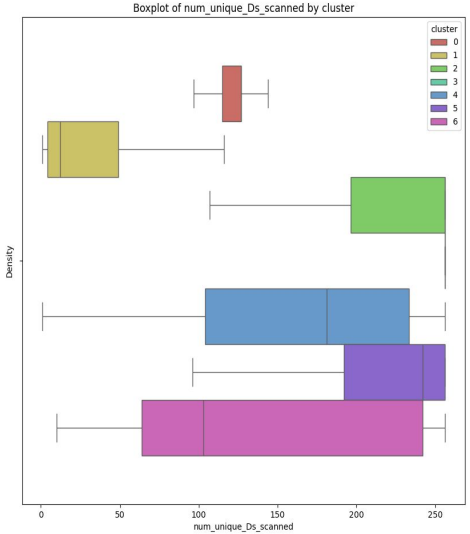
These scanners tend to **scan broadly** with **low number of average packet size**

- Most apparent in cluster 4, 6
- Evenly in all octet and lowest number of packets sent



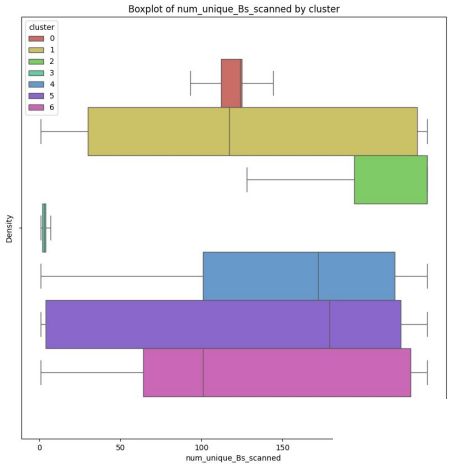


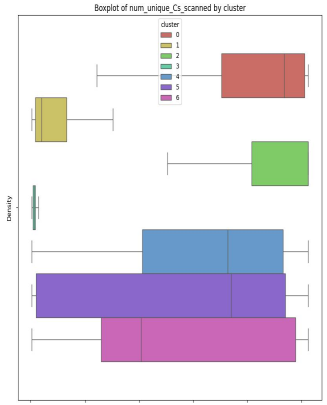


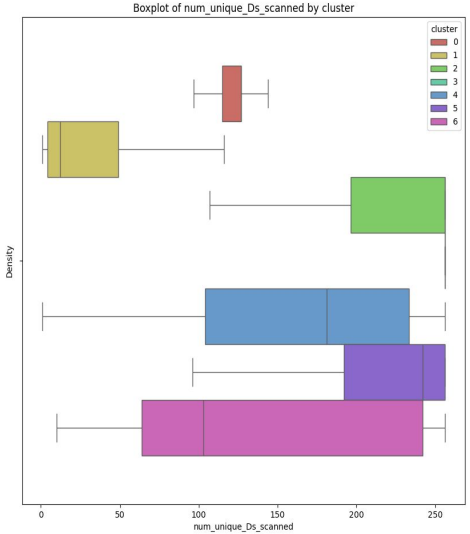


Approach: Subnet-based scanners

- These scanners tend to focus on **scanning a certain octet of an IP** as shown from these figures
- This could be that they discovered a service in a network and are trying to find another service within the same network
- Cluster 1, 0, 3 for second octet, third octet and fourth octet respectively







Approach: UDP-based scanners

These scanners tend to use UDP-based connections to send

- UDP (protocol=6) would require less time and packet size
- Most apparent in cluster 5
- Interestingly, cluster 5 mostly focuses on the fourth octet of an IP
- Implies that UDP is used for **service discovery** after a network portion is discovered to have another service

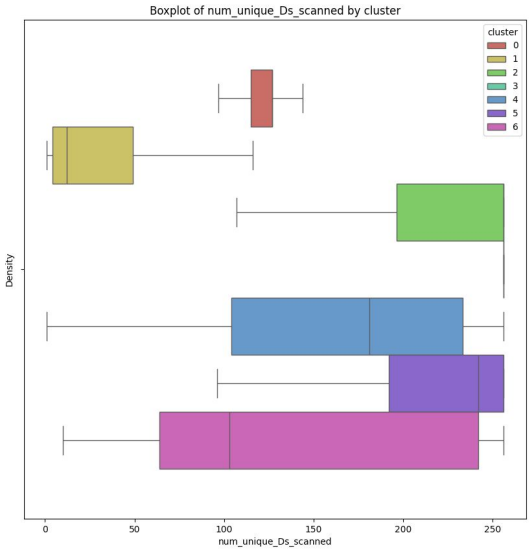


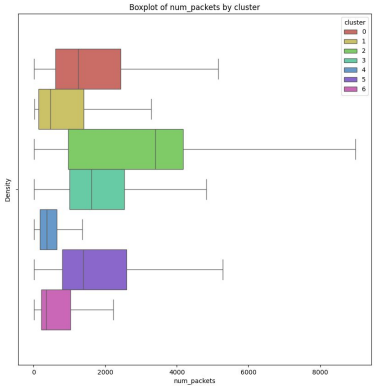
Table 2: Protocol Count by Cluster

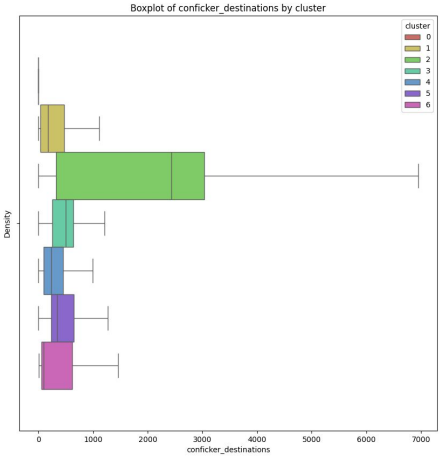
cluster	protocol	count
0	6	12897
0	1	0
0	17	0
1	6	3582
1	17	18
1	1	0
2	6	7090
2	17	1492
2	1	0
3	17	19618
3	1	0
3	6	0
4	1	2224
4	6	6
4	17	2
5	6	83032
5	1	0
5	17	0
6	6	2989
6	17	6
6	1	0

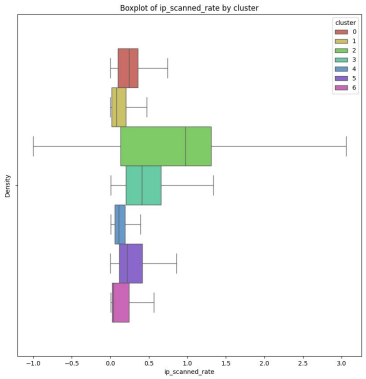
Approach: Conficker Scanner

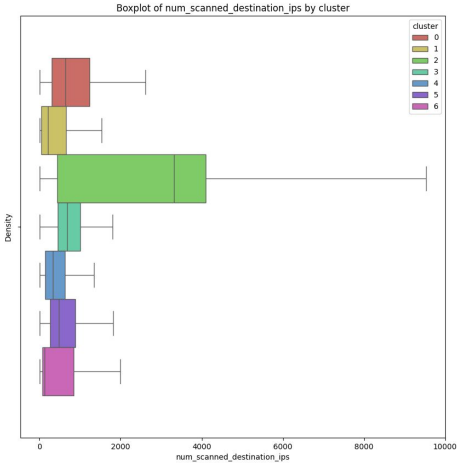
Interestingly, without training for conficker destinations, we found a cluster that scans mainly conficker targeted destinations

- This is apparent in cluster 2
- Scan
  - **most amount of destinations**
  - **most number of packets in an hour**
  - **most number of ips scanned per second**









Benefits: What can they do with this framework?

- Provide a framework using graph neural networks to analyze data own incoming netflows
- Researchers can better target these scanning behaviors, and develop against these behaviors more effectively
- Explore and implement graph-based methods for their own ID systems

Challenges: Limitations and Future Work

Limitations

- Lack of ground truth and/or difficulty in finding ground truth
    - No accuracy metrics
- Constructing graph features/construction is time-consuming and is iterative work
- Hardware is necessary for deep learning, i.e GPU

Future Work/ASK

- Other papers compares clusters similarity between days/hours using metrics like Earth mover's distance to create an automatic detection system with thresholding
- With the success of conficker destinations, try apply methodology to Mirai botnet in September 14th 2016 darknet data to see success of methodology